UNIVERSIDAD DEL VALLE DE GUATEMALA

Proyecto 1 - Kontrol

Avilés, Alejandra – 24722

Figueroa, Ivana – 24785

Montenegro, Juan –24750

Tubac, Jonathán – 24484

Vásquez, Pablo – 24757

Bases de datos I

Guatemala, 2026

Tabla de Contenidos

1. Introducción

2. Docker como Infraestructura como Código

3. Componentes principales de Docker

4. Ventajas de Docker

5. Desventajas y limitaciones

6. Comunidad y ecosistema

7. Docker vs. máquinas virtuales

8. Script de automatización del entorno

9. Estructura de archivos del proyecto

10. Conclusiones

11. Referencias

1. Introducción

Docker es una plataforma de código abierto lanzada en 2013 por Solomon Hykes y Sebastian Pahl, diseñada para facilitar la creación, el despliegue y la ejecución de aplicaciones mediante contenedores. Un contenedor empaqueta una aplicación junto con todas sus dependencias (bibliotecas, archivos de configuración, binarios) en una unidad estandarizada y liviana que se ejecuta de forma consistente en cualquier entorno donde Docker este instalado.

A diferencia de las máquinas virtuales tradicionales, los contenedores Docker comparten el kernel del sistema operativo anfitrión, lo que los hace significativamente más ligeros, rápidos de iniciar y eficientes en el uso de recursos. Esta característica fundamental ha convertido a Docker en una herramienta esencial en el desarrollo de software moderno, DevOps y la práctica de Infraestructura como Código (IaC).

Con más de 13 mil millones de descargas de contenedores por mes y un mercado proyectado a alcanzar los 993 millones de dólares, Docker se ha consolidado como una de las herramientas fundamentales del ecosistema de desarrollo contemporáneo. Empresas como Netflix, Spotify y Airbnb utilizan Docker extensivamente para gestionar sus infraestructuras de microservicios.

2. Docker como Infraestructura como Código

La Infraestructura como Código (IaC) es la práctica de gestionar y aprovisionar infraestructura tecnológica mediante archivos de configuración legibles por máquina, en lugar de procesos manuales. Docker encaja dentro de este paradigma de manera natural.

2.1 El Dockerfile como código de infraestructura

El Dockerfile es un archivo de texto plano que describe, paso a paso, como construir una imagen de contenedor. Define el sistema operativo base, las dependencias a instalar, los archivos de la aplicación a copiar, las variables de entorno, los puertos expuestos y el comando de inicio. Al ser un archivo de texto, puede versionarse en sistemas de control de versiones como Git, revisarse en pull requests y auditarse con facilidad.

2.2 Docker Compose como orquestador local

Docker Compose extiende el concepto de IaC al permitir definir aplicaciones multi-contenedor en un único archivo YAML (docker-compose.yml). En este archivo se describen todos los servicios (contenedores), redes, volúmenes y sus relaciones. Compose puede compararse con Terraform, pero orientado exclusivamente a la gestión de infraestructura de contenedores Docker. Con un solo comando (docker compose up) se puede levantar todo un entorno de desarrollo que incluya frontend, backend y base de datos.

2.3 Principios de IaC que Docker cumple

- **Declarativo:** se describe el estado deseado del entorno, no los pasos para llegar a el.
- **Versionable:** los Dockerfiles y docker-compose.yml se almacenan en repositorios Git.

• **Reproducible:** el mismo archivo genera entornos idénticos en cualquier máquina.

• **Automatizable:** se integra fácilmente en pipelines de CI/CD.

• **Inmutable:** en lugar de modificar contenedores, se reemplazan con nuevas versiones.

• **Documentado:** el archivo de configuración sirve como documentación viva del entorno.

3. Componentes principales de Docker

Componente	Descripción	Función en IaC
Docker Engine	Motor central que ejecuta y gestiona contenedores	Runtime que interpreta las instrucciones de IaC
Dockerfile	Archivo de texto con instrucciones de construcción	Define la infraestructura como código fuente
Docker Images	Plantillas inmutables para crear contenedores	Artefactos versionados de la infraestructura
Docker Containers	Instancias en ejecución de las imágenes	Entornos aislados y reproducibles
Docker Compose	Herramienta para definir apps multi-contenedor	Orquestación declarativa de servicios
Docker Hub	Registro público de imágenes Docker	Repositorio centralizado de imágenes base
Docker Networks	Redes virtuales para comunicación entre contenedores	Configuración de red como código
Docker Volumes	Almacenamiento persistente para contenedores	Gestión de datos persistentes

Tabla 1. Componentes principales de Docker y su rol en IaC.

4. Ventajas de Docker

• **Portabilidad:** los contenedores se ejecutan en cualquier sistema con Docker instalado, independientemente del sistema operativo. Esto elimina el clásico problema de 'funciona en mi maquina' y facilita la migración entre entornos.

• **Eficiencia de recursos:** al compartir el kernel del SO anfitrión, los contenedores son más ligeros que las máquinas virtuales. Tiempos de inicio en segundos, menor consumo de memoria y mayor densidad de aplicaciones por servidor.

• **Consistencia entre entornos:** un nuevo miembro del equipo puede levantar todo el entorno de desarrollo en minutos. Las aplicaciones se comportan idénticamente en desarrollo, staging y producción.

• **Soporte para microservicios:** Docker es ideal para arquitecturas de microservicios, donde cada servicio se ejecuta en su propio contenedor aislado, promoviendo la escalabilidad y facilidad de

mantenimiento.

• **Aislamiento y seguridad:** cada contenedor opera de forma independiente con aislamiento de procesos y sistema de archivos. Esto previene conflictos entre aplicaciones y reduce la superficie de ataque.

• **Reproducibilidad y versionamiento:** los Dockerfiles definen los pasos exactos para construir una imagen. Las imágenes se pueden versionar, facilitando rollbacks y minimizando problemas de despliegue.

• **Integración con CI/CD:** Docker se integra de forma nativa con pipelines de integración y entrega continua, permitiendo automatizar pruebas, compilaciones y despliegues.

• **Ecosistema rico:** Docker Hub ofrece miles de imágenes pre-construidas y oficiales. La comunidad activa y las integraciones con herramientas como Kubernetes amplían sus capacidades.

5. Desventajas y limitaciones

• **Curva de aprendizaje:** aunque los conceptos básicos son accesibles, las configuraciones avanzadas como orquestación, redes y seguridad pueden ser desafiantes, especialmente al integrar Docker en pipelines CI/CD existentes o sistemas legacy.

• **Complejidad de orquestación:** gestionar múltiples contenedores en producción requiere herramientas adicionales como Kubernetes o Docker Swarm, lo que añade complejidad operacional.

• **Overhead de rendimiento:** aunque menor que las VMs, existe una ligera penalización. Aplicaciones que requieren acceso extensivo a GPU o CPU pueden no rendir tan eficientemente como las aplicaciones nativas.

• **Preocupaciones de seguridad:** los contenedores comparten el kernel del SO anfitrión, lo que significa que vulnerabilidades a nivel de kernel pueden afectar múltiples contenedores. Las mis configuraciones pueden ampliar la superficie de ataque.

• **Limitaciones en Windows:** el soporte para contenedores Windows es más limitado comparado con Linux. Las herramientas y ecosistema están principalmente orientados a Linux.

• **Gestión de estado:** los contenedores son por naturaleza efímeros. La persistencia de datos requiere configuración adicional de volúmenes.

• **Consumo de disco:** el sistema de capas puede consumir espacio significativo con el tiempo, especialmente con construcciones frecuentes de imágenes.

6. Comunidad y ecosistema

Docker cuenta con una de las comunidades más activas y grandes en el mundo del desarrollo de software. A continuación, se resumen los aspectos más relevantes de su ecosistema:

Aspecto	**Detalle**
Docker Hub	Registro público con millones de imágenes, incluyendo imágenes oficiales mantenidas por un equipo patrocinado por Docker.
GitHub	Repositorio open source activo con miles de contribuidores y actualizaciones constantes.
Documentación	Documentación oficial extensa y bien organizada en docs.docker.com.
Comunidad	Foros activos, canales de Slack, Docker Community Events y meetups globales.
Integraciones	Compatible con Kubernetes, herramientas CI/CD (Jenkins, GitHub Actions, GitLab CI), clouds (AWS, GCP, Azure).
Docker Desktop	Herramienta gratuita para desarrollo local en Windows, macOS y Linux.
Alternativas	Podman, containerd y LXD como opciones complementarias o alternativas.

Tabla 2. Ecosistema y comunidad de Docker.

7. Docker vs. máquinas virtuales

La siguiente tabla presenta una comparación directa entre contenedores Docker y máquinas virtuales tradicionales:

Característica	Docker (Contenedores)	Máquinas Virtuales
Virtualización	A nivel de SO	A nivel de hardware
Tiempo de inicio	Segundos	Minutos
Tamaño típico	Megabytes	Gigabytes
Rendimiento	Cercano al nativo	Overhead significativo
Aislamiento	A nivel de proceso	Completo (SO separado)
Portabilidad	Alta	Media
Densidad	Miles por host	Decenas por host
SO requerido	Comparte kernel del host	SO completo por VM
Uso de RAM	Bajo	Alto
Ideal para	Microservicios, CI/CD	Apps legacy, SO distinto

Tabla 3. Comparativa entre Docker y máquinas virtuales.

8. Script de automatización del entorno

A continuación, se presenta la configuración Docker completa para automatizar el entorno de desarrollo del proyecto, que utiliza Vue.js para el frontend, Go con el framework Gin para el backend, y PostgreSQL como motor de base de datos.

8.1 docker-compose.yml

```
services:
  db:
    image: postgres:16-alpine
    container_name:proyecto_db
    restart: unless-stopped
    environment:
      POSTGRES_USER: ${DB_USER:-admin}
      POSTGRES_PASSWORD:${DB_PASSWORD:-secret123}
      POSTGRES_DB:${DB_NAME:-proyecto_db}
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data
      - ./db/init.sql:/docker-entrypoint-initdb.d/init.sql
    networks:
      - app-network
```

Este archivo define y orquesta los tres servicios del proyecto. Con un solo comando (docker compose up) se levanta todo el entorno:

```yaml
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U admin"]
      interval: 5s
      timeout: 5s
      retries: 5

  backend:
    build:
      context: ./backend
      dockerfile: Dockerfile
    container_name: proyecto_backend
    restart: unless-stopped
    environment:
      DB_HOST: db
      DB_PORT: 5432
      DB_USER: ${DB_USER:-admin}
      DB_PASSWORD: ${DB_PASSWORD:-secret123}
      DB_NAME: ${DB_NAME:-proyecto_db}
      GIN_MODE: debug
      PORT: 8080
    ports:
      - "8080:8080"
    depends_on:
      db:
        condition: service_healthy
    networks:
      - app-network

  frontend:
    build:
      context: ./frontend
      dockerfile: Dockerfile
    container_name: proyecto_frontend
    restart: unless-stopped
    environment:
      VITE_API_URL: http://localhost:8080
    ports:
      - "3000:3000"
    depends_on:
      - backend
    networks:
      - app-network

volumes:
  pgdata:

networks:
  app-network:
    driver: bridge
```

8.2 Dockerfile del backend (Go + Gin)

Se

```
# Etapa 1: Compilacion
FROM golang:1.23-alpine AS builder
WORKDIR /app
COPY go.mod go.sum ./
RUN go mod download
COPY . .
RUN CGO_ENABLED=0 GOOS=linux go build -o main ./cmd/main.go

# Etapa 2: Imagen final
FROM alpine:3.19
RUN apk --no-cache add ca-certificates
WORKDIR /app
COPY --from=builder /app/main .
EXPOSE 8080
CMD ["./main"]
```

Utiliza un build multi-stage para producir una imagen final mínima. La primera etapa compila el binario de Go y la segunda copia solo el ejecutable a una imagen Alpine ligera:

8.3 Dockerfile del frontend (Vue.js)

```
# Etapa 1: Build
FROMnode:20-alpineASbuilder
WORKDIR /app
COPYpackage*.json./
RUN npm ci
COPY . .
RUN npm run build

# Etapa 2: Servir
FROMnode:20-alpine
RUNnpminstall-gserve
WORKDIR /app
COPY--from=builder/app/dist./dist
EXPOSE 3000
CMD ["serve", "-s", "dist", "-l", "3000"]
```

También se usa multi-stage build. La primera etapa instala dependencias y construye los archivos estáticos, y la segunda los sirve con un servidor ligero:

8.4 Archivo .env

```
DB_USER=admin
DB_PASSWORD=secret123
DB_NAME=proyecto_db
```

8.5 Comandos para ejecutar el entorno

```
#Construirylevantartodoslosservicios docker
compose up --build

# Levantar en segundo plano
docker compose up --build -d

# Ver logs
docker compose logs -f

# Detener todos los servicios
```

```
docker compose down

#Deteneryeliminarvolumenes(resetearBD)
docker compose down -v
```

9. Estructura de archivos del proyecto

Esta estructura sigue las mejores prácticas de Docker, separando cada servicio en su propio directorio con su Dockerfile correspondiente. El archivo docker-compose.yml en la raíz del proyecto actúa como el punto central de configuración que conecta todos los servicios.

Al ejecutar docker compose up --build, Docker automáticamente: construye la imagen del backend compilando el código Go, construye la imagen del frontend instalando dependencias npm y ejecutando el build de Vue.js, descarga la imagen oficial de PostgreSQL 16, crea una red virtual para que los tres servicios se comuniquen, crea un volumen persistente para los datos de PostgreSQL, y inicia los servicios en el orden correcto respetando las dependencias definidas.

10. Conclusiones

Docker es una herramienta poderosa para IaC.

Permite definir entornos de desarrollo, testing y producción de forma declarativa a través de archivos de configuración versionables. Esto cumple con los principios fundamentales de la Infraestructura como Código: reproducibilidad, automatización y consistencia.

La portabilidad es el mayor diferenciador.

La capacidad de empaquetar una aplicación con todas sus dependencias garantiza que el entorno funcione idénticamente en cualquier máquina, eliminando el problema histórico de inconsistencias entre desarrollo y producción.

Docker Compose simplifica entornos complejos.

Para proyectos con múltiples servicios como el nuestro (Vue.js + Go/Gin + PostgreSQL), Docker Compose permite orquestar todo el stack con un solo comando, reduciendo drásticamente el tiempo de configuración.

Los multi-stage builds optimizan las imágenes.

El uso de builds multi-etapa permite compilar y empaquetar aplicaciones en imágenes finales mínimas, mejorando tiempos de descarga, despliegue y uso de recursos.

La curva de aprendizaje vale la pena.

Aunque Docker requiere una inversión inicial de tiempo para aprender sus conceptos, los beneficios a largo plazo en productividad, consistencia y automatización superan ampliamente el costo de aprendizaje.

Docker sigue siendo relevante en 2025.

A pesar de la aparición de alternativas como Podman y containerd, Docker mantiene su posición como la herramienta de contenerizacion más popular y accesible, con una comunidad activa y un ecosistema robusto.

11. Referencias

Docker Inc. (2025). Docker Documentation. https://docs.docker.com/

Docker Inc. (2025). Docker Compose Overview. https://docs.docker.com/compose/

Docker Inc. (2025). Dockerfile Reference. https://docs.docker.com/reference/dockerfile/

Docker Inc. (2025). Docker Hub. https://hub.docker.com/

Rancher Labs. Containers: Making Infrastructure as Code Easier
https://www.rancher.cn/containers-making-infrastructure-code-easier

XenonStack. (2024). Infrastructure as Code and Containers: Better Together.
https://www.xenonstack.com/insights/infrastructure-as-code-containers

Red Badger. Using Docker Containers for Immutable Infrastructure as Code.
https://content.red-badger.com/resources/using-docker-containers-for-immutable-infrastructure-as-code

DuploCloud. (2025). Docker Advantages and Disadvantages.
https://duplocloud.com/blog/docker-advantages-and-disadvantages/